

Diseño de integración de Gentle-AI en InvestigIA

Adaptación de Engram, Skills, Contratos y Flujo SDD al ecosistema multiagente de InvestigIA

Centro de Investigación en Computación, IPN

17 de junio de 2026

1 Introducción

InvestigIA transforma un perfil de estudiante de posgrado en un protocolo preliminar de investigación mediante seis agentes especializados: perfilado, matching, metodológico, investigador, redacción y revisión. El notebook del proyecto establece que cada agente debe cumplir con contrato de datos, prompt versionado, máximo de iteraciones, manejo de errores, logging y pruebas.

El problema es que esos requisitos quedan enunciados pero no diseñados. ¿Qué pasa exactamente cuando el agente de perfil genera un JSON incompleto? ¿Cómo sabe el agente de redacción qué evidencia recuperó el investigador? ¿Con qué criterio se decide si el protocolo final es suficiente o necesita otra ronda de correcciones?

Este documento propone una respuesta concreta tomando los conceptos de Gentle-AI (memoria persistente, flujo estructurado con validación, instrucciones especializadas por contexto) y adaptándolos a la arquitectura que ya define InvestigIA. No se trata de instalar Gentle-AI como dependencia externa, sino de implementar las mismas ideas directamente en el código del proyecto.

2 Flujo multiagente con puertas de validación

El pipeline de InvestigIA no debería ser lineal. Si el agente de perfil genera un JSON con la lista de intereses vacía, no tiene sentido pasar eso al agente de matching. Lo razonable es validar la salida de cada agente antes de permitirle avanzar al siguiente.

A esas validaciones les llamamos puertas. Cada puerta V_i revisa que la salida de un agente cumpla un contrato mínimo antes de pasarla al siguiente nodo del flujo.

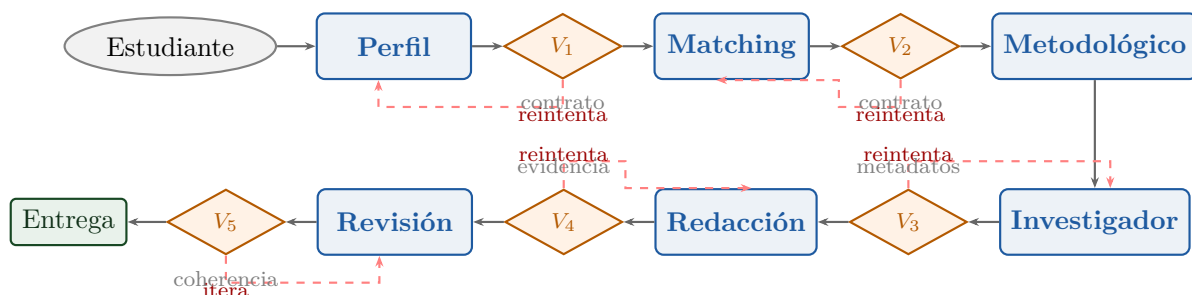


Figura 1: Pipeline multiagente con puertas de validación y caminos de reintento

Las reglas de cada puerta:

V_1 : Perfil $\rightarrow$ Matching

El perfil JSON debe tener nombre no vacío, nivel académico válido (licenciatura, maestría o doctorado), y al menos un interés. Si falta cualquiera de estos campos, el agente de perfil vuelve a preguntarle al estudiante.

V_2 : Matching $\rightarrow$ Metodológico

El ranking debe contener al menos un resultado con score mayor a cero. Un matching vacío significa que los intereses del estudiante no coinciden con ninguna línea o asesor disponible; en ese caso se le pide que amplíe o reformule.

V_3 : Metodológico $\rightarrow$ Investigador

El esquema de investigación necesita pregunta de investigación, al menos un objetivo específico y una estrategia metodológica explícita. Si el agente metodológico no logra definir alguno de estos elementos, itera con las observaciones que resulten del intento fallido.

V_4 : Investigador $\rightarrow$ Redacción

Cada fragmento recuperado debe incluir identificador, título, autores, año y texto. Los fragmentos que no cumplan se descartan. Si después del filtrado no queda ninguno, la evidencia se marca como insuficiente y el agente de redacción lo recibe como input explícito.

V_5 : Revisión $\rightarrow$ Entrega

El agente de revisión evalúa el protocolo contra reglas deterministas y genera un puntaje de coherencia entre 0 y 1. Si el puntaje cae por debajo del umbral (0.7 en la primera implementación), el protocolo regresa a redacción junto con las observaciones específicas. Si pasa, se entrega al estudiante.

La idea de las puertas viene del concepto de verificación en SDD: cada fase debe confirmarse antes de avanzar. En un sistema multiagente esto es particularmente útil porque un error en un nodo temprano se propaga a todos los posteriores.

3 Engram como capa de estado persistente

Gentle-AI usa Engram para guardar decisiones de arquitectura entre sesiones de coding. En InvestigIA el concepto se adapta para algo distinto: mantener el estado completo de la interacción con el estudiante.

La diferencia es sutil pero importante. En Gentle-AI, Engram guarda decisiones que tomó el desarrollador. En InvestigIA, Engram guarda el resultado del trabajo que produjeron los agentes para un estudiante específico.

3.1 Qué se persiste

Dato		Topic key	Quién lo escribe
Perfil estudiante	del	session/id/perfil	Agente de Perfil
Líneas seleccionadas		session/id/matching	Agente de Matching
Esquema investigación	de	session/id/esquema	Agente Metodológico
Evidencia recuperada		session/id/evidencia	Agente Investigador
Protocolo generado		session/id/protocolo	Agente de Redacción
Reporte de revisión		session/id/revision	Agente de Revisión
Historial iteraciones	de	session/id/iteraciones	Orquestador

Cuadro 1: Estructura de persistencia en Engram

3.2 Para qué sirve

Recuperación de sesión. Si el estudiante cierra la aplicación a la mitad del proceso y regresa horas después, el sistema retoma exactamente donde quedó. El perfil, la evidencia ya recuperada, el esquema metodológico, todo sigue ahí. Sin esta persistencia, cada sesión empieza de cero.

Contexto entre agentes. Cuando el agente de redacción necesita generar el protocolo, consulta directamente en Engram la evidencia recuperada y el esquema metodológico. El orquestador no tiene que pasar toda esa información como parámetro en cada llamada. Los agentes leen lo que necesitan de la capa de estado compartido.

Auditoría. Cada iteración queda registrada con timestamp y resultado. Si el agente de revisión rechaza el protocolo tres veces seguidas, el historial permite analizar qué cambió entre cada intento. ¿Mejoró la redacción? ¿Se agregaron fuentes? ¿O el agente simplemente repite los mismos errores?

Métricas. El conteo de iteraciones por agente responde directamente a la pregunta de investigación PI-5. Si el agente metodológico necesita en promedio 2.3 iteraciones para producir un esquema válido, mientras que el de perfil pasa en 1.1, eso indica dónde el sistema tiene más dificultades y qué componentes aportan menos valor del esperado.

3.3 Modelo de datos

Cada observación en Engram sigue esta estructura:

```
{
  "topic_key": "session/abc123/perfil",
  "type": "state",
  "content": {
    "nombre": "María López",
    "nivel": "maestria",
    "intereses": ["robótica", "visión por computadora"],
    "experiencia": "proyecto de detección de objetos",
    "timestamp": "2026-06-17T10:30:00Z"
  },
  "session_id": "abc123"
}
```

4 Skills por agente

En Gentle-AI, un skill es un conjunto de instrucciones que el agente de coding carga cuando el contexto de trabajo lo requiere. En InvestigIA se aplica la misma idea, pero las instrucciones definen el comportamiento de cada agente del sistema.

Los skills no son prompts. El prompt es lo que se envía al LLM en cada llamada. El skill es la capa encima: las reglas, validaciones y patrones que el agente debe seguir independientemente del prompt específico que reciba en una iteración determinada.

4.1 Agente de Perfil

perfil/estructura

Define el esquema JSON del perfil con sus campos obligatorios y opcionales. La validación se hace con Pydantic: intereses mínimo 1 elemento, nombre no vacío, nivel restringido a tres valores posibles.

perfil/preguntas-adaptativas

Lógica para generar preguntas de seguimiento cuando el perfil está incompleto. Si el estudiante no menciona experiencia previa, el agente pregunta directamente por eso. Si los intereses son demasiado amplios (“inteligencia artificial” sin más), pide que delimite.

4.2 Agente de Matching

matching/scoring

La función de puntuación del notebook: $S(a_i, s) = w_1 I_i + w_2 E_i + w_3 H_i + w_4 D_i$. Los pesos w_j son configurables y deben sumar 1. En la PoC se puede empezar con pesos iguales (0.25 cada uno) y ajustar según los resultados de los casos de prueba.

matching/explainability

Cada resultado del ranking incluye una explicación textual. El estudiante

necesita saber *por qué* un asesor es candidato, no solo que aparece primero en la lista. Sin explicación, el matching es una caja negra.

4.3 Agente Metodológico

metodologico/delimitacion

Flujo guiado para construir el esquema de investigación: problema, alcance, pregunta, objetivos (general y específicos), hipótesis. Cada sección tiene sus propias reglas de validación interna.

metodologico/coherencia-interna

Reglas deterministas que se evalúan antes de pasar al siguiente agente. La pregunta de investigación debe conectarse con el objetivo general. Los objetivos específicos deben ser alcanzables con la metodología propuesta. La hipótesis, si existe, debe ser testeable.

4.4 Agente Investigador (RAG)

investigador/retrieval

Pipeline de recuperación: a partir del esquema de investigación se genera una consulta expandida, se calcula similitud coseno sobre los embeddings del corpus, y se recuperan los 5 fragmentos más relevantes. Cada fragmento conserva sus metadatos completos.

investigador/metadatos

Validación estricta de metadatos. Identificador, título, autores, año y texto son obligatorios. Un fragmento sin autores o con año vacío se descarta sin excepción. Si todos se descartan, la evidencia se marca como insuficiente.

investigador/citas

El agente no puede generar citas que no existan en los fragmentos recuperados. Esta es probablemente la regla más importante del sistema: si el LLM necesita apoyar una afirmación y no hay evidencia disponible, debe marcarlo explícitamente como **[sin fuente]** en lugar de inventar una referencia.

4.5 Agente de Redacción

redaccion/protocolo

Plantilla del protocolo con sus secciones mínimas: pregunta, objetivos, hipótesis, metodología, justificación, alcance, resultados esperados. Cada sección se genera condicionada por la evidencia que recuperó el agente investigador.

redaccion/separacion-evidencia

Cada afirmación del protocolo debe distinguir entre lo que proviene de la evidencia recuperada y lo que el LLM generó por su cuenta. Las afirmaciones sin fuente se marcan como **[sin fuente]**. El estudiante y el asesor deben poder diferenciar a simple vista qué tiene respaldo documental y qué no.

4.6 Agente de Revisión

revision/reglas-deterministas

Lista de verificación automática: ¿la pregunta conecta con el objetivo general? ¿Los objetivos específicos son medibles? ¿La metodología es compatible con el tipo de estudio? ¿Las referencias citadas existen en el corpus recuperado?

revision/puntaje

Genera un puntaje de coherencia entre 0 y 1. El umbral de aprobación inicial es 0.7, aunque habría que calibrarlo con los casos de prueba C1 a C6 del diseño experimental.

revision/iteracion

Máximo 3 intentos. Si después de tres revisiones el puntaje sigue por debajo del umbral, se entrega el protocolo con las observaciones pendientes y una nota indicando que requiere revisión humana. Es preferible entregar algo imperfecto con las fallas documentadas que entrar en un bucle infinito.

5 Contratos entre agentes

Cada transición entre agentes tiene un contrato definido con Pydantic. La idea es simple: si la salida de un agente no cumple el contrato que espera el siguiente, el orquestador captura el error y itera en lugar de propagar datos inválidos por todo el pipeline.

5.1 Contrato del agente de perfil

```
1 from pydantic import BaseModel, Field
2 from typing import List
3 from enum import Enum
4
5 class NivelAcademico(str, Enum):
6     licenciatura = "licenciatura"
7     maestria = "maestria"
8     doctorado = "doctorado"
9
10 class PerfilEstudiante(BaseModel):
11     nombre: str = Field(..., min_length=1)
12     nivel: NivelAcademico
13     intereses: List[str] = Field(..., min_items=1)
14     experiencia_previa: str | None = None
15     institucion: str | None = None
```

Cuando el LLM genera un JSON con `intereses: []`, Pydantic levanta un error de validación. El orquestador captura ese error, le pregunta al estudiante qué líneas de investigación le interesan, y vuelve a invocar el agente de perfil con la respuesta.

5.2 Contrato del agente investigador

```
1 from pydantic import BaseModel, Field
2
3 class FragmentoRecuperado(BaseModel):
4     id: str
5     titulo: str = Field(..., min_length=1)
6     autores: List[str] = Field(..., min_items=1)
7     anio: int = Field(..., ge=1900, le=2030)
8     texto: str = Field(..., min_length=50)
9     score: float = Field(..., ge=0.0, le=1.0)
10
11 class EvidenciaRecuperada(BaseModel):
12     fragmentos: List[FragmentoRecuperado]
13     evidencia_insuficiente: bool = False
14
15     def tiene_evidencia_valida(self) -> bool:
16         return len(self.fragmentos) > 0
```

Los fragmentos sin autores o con año fuera de rango se descartan en la validación. Si no queda ninguno, `evidencia_insuficiente` se marca como `True`. El agente de redacción recibe esa señal y, en lugar de inventar referencias, genera el protocolo indicando qué secciones carecen de respaldo documental.

5.3 Contrato de revisión

```
1 from pydantic import BaseModel, Field
2 from typing import List
3
4 class Observacion(BaseModel):
5     seccion: str
6     tipo: str # "error" | "warning" | "sugerencia"
7     descripcion: str
8
9 class ReporteRevision(BaseModel):
10     puntaje_coherencia: float = Field(..., ge=0.0, le=1.0)
11     observaciones: List[Observacion]
12     aprobado: bool
13
14     @property
15     def necesita_iteracion(self) -> bool:
16         return not self.aprobado
```

El orquestador lee `necesita_iteracion`. Cuando es `True`, pasa las observaciones al agente de redacción como contexto adicional. El agente de redacción entonces sabe qué secciones fallaron y por qué, lo cual es más útil que simplemente volver a intentar sin información sobre el error anterior.

6 Orquestación con LangGraph

LangGraph modela el flujo como un grafo dirigido donde cada nodo es un agente y cada arista es una puerta de validación. Las aristas condicionales permiten que el grafo decida en tiempo de ejecución si avanza o itera.

```

1 from langgraph.graph import StateGraph
2
3 # Estado compartido entre todos los nodos
4 class EstadoInvestigIA(TypedDict):
5     perfil: PerfilEstudiante | None
6     matching: MatchingResultado | None
7     esquema: EsquemaInvestigacion | None
8     evidencia: EvidenciaRecuperada | None
9     protocolo: Protocolo | None
10    revision: ReporteRevision | None
11    iteraciones: dict # conteo por agente
12
13 # Grafo
14 grafo = StateGraph(EstadoInvestigIA)
15
16 grafo.add_node("perfil", agente_perfil)
17 grafo.add_node("matching", agente_matching)
18 grafo.add_node("metodologico", agente_metodologico)
19 grafo.add_node("investigador", agente_investigador)
20 grafo.add_node("redaccion", agente_redaccion)
21 grafo.add_node("revision", agente_revision)
22
23 # Aristas condicionales: validación
24 grafo.add_conditional_edges(
25     "perfil",
26     validar_perfil,          # ¿cumple contrato?
27     {"ok": "matching", "retry": "perfil"}
28 )
29 grafo.add_conditional_edges(
30     "revision",
31     validar_protocolo,      # ¿puntaje >= 0.7?
32     {"ok": END, "retry": "redaccion"}
33 )

```

Cada función de validación (como `validar_perfil` o `validar_protocolo`) evalúa el contrato Pydantic correspondiente y devuelve “ok” o “retry”. El grafo no necesita lógica condicional escrita a mano; LangGraph se encarga del enrutamiento.

6.1 Límite de iteraciones

El notebook establece “máximo de iteraciones” como requisito de cada agente. La implementación propone 3 intentos como límite. Cuando un agente falla tres veces, el sistema registra las tres salidas y sus errores en Engram, marca esa etapa como `requiere_intervencion` y continúa con el siguiente agente si el flujo lo permite.

Esto evita que el sistema se quede atascado en un bucle sin salida. En un sprint de 10

días, es prefericiente detectar dónde el LLM no puede solo y documentar la limitación, en vez de gastar tiempo ajustando prompts para un caso que tal vez requiera intervención humana de todas formas.

7 Integración con LM Studio

Los seis agentes se comunican con Gemma 4 a través de LM Studio usando la interfaz compatible con OpenAI. El cliente se instancia una vez y se comparte:

```
1 from openai import OpenAI
2 import os
3
4 cliente = OpenAI(
5     base_url=os.getenv("LLM_BASE_URL", "http://localhost:1234/v1"),
6     api_key=os.getenv("LLM_API_KEY", "lm-studio"),
7 )
8
9 def invocar_agente(
10     sistema: str,
11     usuario: str,
12     modelo: str = "gemma-4",
13     temperatura: float = 0.2,
14 ) -> str:
15     respuesta = cliente.chat.completions.create(
16         model=modelo,
17         temperature=temperatura,
18         messages=[
19             {"role": "system", "content": sistema},
20             {"role": "user", "content": usuario},
21         ],
22     )
23     return respuesta.choices[0].message.content
```

Cada skill define su propio prompt de sistema. La temperatura varía según la tarea: 0.1 o 0.2 para perfil y revisión, donde se busca determinismo; 0.4 o 0.6 para redacción, donde cierta variabilidad mejora la calidad del texto generado.

8 Mapeo con las preguntas de investigación

El diseño propuesto no es independiente de las preguntas de investigación del notebook. Cada componente responde a una o más preguntas de forma directa.

La pregunta PI-3 sobre latencias es la que menos se aborda en este documento. La asignación de Gemma 4 como modelo principal ayuda, pero las latencias reales dependen del hardware, la cuantización y el tamaño del corpus. Eso habría que medirlo en la PoC, no se puede resolver en el diseño.

PI	Pregunta	Cómo se aborda
PI-1	Multiagente transforma información no estructurada en protocolo	Flujo con 6 agentes y contratos Pydantic
PI-2	Trazabilidad bibliográfica con RAG local	Fragmentos con metadatos obligatorios, citas solo del corpus
PI-3	Latencias compatibles con interacción	Gemma 4 para todo excepto verificación
PI-4	Validaciones deterministas + LLM	Puertas V_i con reglas Pydantic + revisión por LLM
PI-5	Qué componentes aportan más valor	Engram registra iteraciones por agente, se puede medir

Cuadro 2: Relación entre preguntas de investigación y diseño

9 Limitaciones

Gemma 4 y validación estricta. Los contratos Pydantic asumen que el LLM genera JSON válido. Un modelo cuantizado puede producir JSON malformado, especialmente en respuestas largas. La mitigación es combinar reintentos con un prompt de corrección y un parser tolerante que intente reparar errores menores (comas finales, comillas simples en vez de dobles).

Dependencia de Engram. Si Engram no está disponible en el entorno de despliegue, el sistema necesita un fallback. La opción más simple es almacenamiento local: un archivo JSON por sesión en disco, o una tabla SQLite. Funciona, pero pierde la ventaja de la memoria compartida entre sesiones.

Umbral de revisión. El puntaje 0.7 para aprobar el protocolo es una primera aproximación. Podría ser demasiado permisivo o demasiado estricto; no hay forma de saberlo sin ejecutar los casos de prueba C1 a C6 y ver qué puntajes producen. Es un hiperparámetro que se calibra en la práctica.

Costo de iteración. Cada intento adicional es una llamada extra a Gemma 4. Si un agente falla tres veces, eso son tres llamadas que no estaban en el flujo ideal. En el hardware institucional, cada llamada puede tomar entre 10 y 30 segundos dependiendo de la longitud del prompt. No es crítico, pero sí se nota en un sprint de 10 días.

Skills como código. Los skills descritos aquí son descripciones conceptuales. Para que el orquestador los cargue dinámicamente, hay que implementarlos como archivos con un formato estándar (el mismo que usa Gentle-AI para sus skills). Esa implementación no está hecha todavía.

10 Próximos pasos

1. Definir los esquemas Pydantic completos para los 6 agentes.
2. Implementar el grafo LangGraph con las aristas condicionales.

3. Escribir los prompts de sistema de cada agente (primer versión).
4. Configurar Engram con la estructura de topic keys propuesta.
5. Probar con el caso C1 (estudiante con tema definido) del diseño experimental.
6. Calibrar el umbral de revisión con los casos C1 a C6.

Referencias

- [1] Equipo InvestigIA, *Plan de acción para el desarrollo de InvestigIA*, Notebook Jupyter, CIC-IPN, 2026.
- [2] Gentleman Programming, *Gentle-AI – Ecosystem, Frameworks, Workflows for AI coding agents*, <https://github.com/Gentleman-Programming/gentle-ai>, 2026.
- [3] Gentleman Programming, *Engram Commands – Gentle-AI*, <https://github.com/Gentleman-Programming/gentle-ai/blob/main/docs/engram.md>, 2026.
- [4] LangChain, *LangGraph – Building agent state machines*, <https://github.com/langchain-ai/langgraph>, 2025.